

LAPORAN PRAKTIKUM

STRUKTUR DATA

SORTING

PEKAN 7



Disusun Oleh :

KINAYA NOVRYA MANDA

(2511531016)

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

MUHAMMAD GALID AVERO

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis sampaikan kehadirat Allah SWT. Salawat dan salam disampaikan kepada Nabi Muhammad SAW. Karena thaufik dan hidayah-Nya, laporan praktikum ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban dari kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar serta penerapan bahasa pemrograman Java.

Melalui praktikum ini, penulis memperoleh pengalaman langsung dalam memahami Implementasi Sorting menggunakan Java. Diharapkan laporan ini dapat memberikan gambaran yang jelas mengenai materi yang telah dipelajari serta hasil dari percobaan yang dilakukan selama praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi isi maupun penyajiannya. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu, asisten laboratorium, serta semua pihak yang telah memberikan bimbingan, arahan, dan dukungan sehingga laporan praktikum ini dapat tersusun. Semoga laporan ini dapat bermanfaat bagi pembaca, khususnya dalam memahami dasar-dasar pemrograman Java.

Padang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
DAFTAR LAMPIRAN.....	iii
BAB 1 PENDAHULUAN	
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	2
1.3 Manfaat Praktikum.....	2
BAB 2 PEMBAHASAN	
2.1 Dasar Teori.....	3
2.2 Kode Program.....	6
2.3 Langkah Kerja.....	10
2.4 Analisis Hasil.....	26
BAB 3 KESIMPULAN	
3.1 Hasil Praktikum.....	27
3.2 Saran Pengembangan.....	30
DAFTAR KEPUSTAKAAN.....	31
TUGAS PRAKTIKUM PEKAN 7.....	32

DAFTAR LAMPIRAN

Kode Program 2.1 (Kode Program InsertionSort).....	6
Kode Program 2.2 (Kode Program SelectionSort).....	6
Kode Program 2.3 (Kode Program BubbleSort).....	7
Kode Program 2.4 (Kode Program InsertionSortGUI).....	9
Gambar 2.1 (Langkah 1 InsertionSort).....	10
Gambar 2.2 (Langkah 2 InsertionSort).....	10
Gambar 2.3 (Langkah 3 InsertionSort).....	11
Gambar 2.4 (Langkah 4 InsertionSort).....	11
Gambar 2.5 (Langkah 5 InsertionSort).....	11
Gambar 2.6 (Langkah 6 InsertionSort).....	12
Gambar 2.7 (Langkah 7 InsertionSort).....	12
Gambar 2.8 (Langkah 1 SelectionSort).....	12
Gambar 2.9 (Langkah 2 SelectionSort).....	13
Gambar 2.10 (Langkah 3 SelectionSort).....	13
Gambar 2.11 (Langkah 4 SelectionSort).....	13
Gambar 2.12 (Langkah 5 SelectionSort).....	14
Gambar 2.13 (Langkah 6 SelectionSort).....	14
Gambar 2.14 (Langkah 7 SelectionSort).....	14
Gambar 2.15 (Langkah 1 BubbleSort).....	15
Gambar 2.16 (Langkah 2 BubbleSort).....	15
Gambar 2.17 (Langkah 3 BubbleSort).....	16
Gambar 2.18 (Langkah 4 BubbleSort).....	16
Gambar 2.19 (Langkah 5 BubbleSort).....	16

Gambar 2.20 (Langkah 6 BubbleSort).....	17
Gambar 2.21 (Langkah 7 BubbleSort).....	17
Gambar 2.22 (Langkah 1 InsertionSortGUI).....	17
Gambar 2.23 (Langkah 2 InsertionSortGUI).....	18
Gambar 2.24 (Langkah 3 InsertionSortGUI).....	18
Gambar 2.25 (Langkah 4 InsertionSortGUI).....	19
Gambar 2.26 (Langkah 5 InsertionSortGUI).....	20
Gambar 2.27 (Langkah 6 InsertionSortGUI).....	20
Gambar 2.28 (Langkah 7 InsertionSortGUI).....	20
Gambar 2.29 (Langkah 8 InsertionSortGUI).....	21
Gambar 2.30 (Langkah 9 InsertionSortGUI).....	21
Gambar 2.31 (Langkah 10 InsertionSortGUI).....	21
Gambar 2.32 (Langkah 11 InsertionSortGUI).....	22
Gambar 2.33 (Langkah 12 InsertionSortGUI).....	22
Gambar 2.34 (Langkah 13 InsertionSortGUI).....	23
Gambar 2.35 (Langkah 14 InsertionSortGUI).....	23
Gambar 2.36 (Langkah 15 InsertionSortGUI).....	24
Gambar 2.37 (Langkah 16 InsertionSortGUI).....	24
Gambar 2.38 (Langkah 17 InsertionSortGUI).....	24
Gambar 2.39 (Langkah 18 InsertionSortGUI).....	25
Gambar 2.40 (Langkah 19 InsertionSortGUI).....	25
Gambar 2.41 (Langkah 20 InsertionSortGUI).....	26
Gambar 3.1 (Hasil Praktikum InsertionSort).....	27
Gambar 3.2 (Hasil Praktikum SelectionSort).....	27
Gambar 3.3 (Hasil Praktikum BubbleSort).....	28
Gambar 3.4 (Hasil Praktikum InsertionSortGUI).....	29

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Pengurutan data atau sorting merupakan salah satu konsep dasar dalam pemrograman yang sangat sering digunakan dalam pengolahan data. Dalam banyak kasus, data yang tersusun rapi akan lebih mudah dicari, dianalisis, maupun diproses lebih lanjut. Karena itu, memahami cara kerja algoritma sorting menjadi hal penting bagi mahasiswa yang sedang mempelajari struktur data dan algoritma. Pada praktikum ini digunakan beberapa metode pengurutan sederhana seperti Insertion Sort, Selection Sort, dan Bubble Sort untuk melihat bagaimana proses pengurutan dilakukan secara bertahap di dalam program.

Setiap algoritma sorting memiliki cara kerja yang berbeda walaupun tujuan akhirnya sama, yaitu menyusun data dari nilai terkecil ke terbesar. Insertion Sort bekerja dengan menyisipkan elemen ke posisi yang sesuai pada bagian array yang sudah terurut, Selection Sort memilih elemen terkecil pada setiap tahap lalu menukarnya ke posisi yang benar, sedangkan Bubble Sort melakukan pertukaran antar elemen yang berdekatan secara berulang. Perbedaan mekanisme ini membuat praktikum menjadi lebih menarik karena mahasiswa dapat memahami logika pengurutan dari berbagai sudut pandang, bukan hanya melihat hasil akhirnya saja.

Selain menggunakan program berbasis console, praktikum juga dilengkapi dengan tampilan GUI pada Insertion Sort untuk memperlihatkan proses pengurutan secara visual. Dengan adanya visualisasi langkah demi langkah, proses perpindahan dan perbandingan data menjadi lebih mudah dipahami. Hal ini membantu mahasiswa melihat bagaimana algoritma bekerja secara nyata di dalam program, sehingga pembelajaran tidak hanya bersifat teori tetapi juga dapat diamati langsung melalui proses yang berjalan pada aplikasi.

1.2 Tujuan Praktikum

Tujuan dilakukannya praktikum ini adalah sebagai berikut :

1. Memahami konsep dasar algoritma sorting menggunakan metode Insertion Sort, Selection Sort, dan Bubble Sort.
2. Mengetahui perbedaan proses pengurutan data pada masing-masing algoritma sorting yang digunakan dalam program.
3. Melatih kemampuan mahasiswa dalam membuat program pengurutan data serta menampilkan proses sorting melalui tampilan console dan GUI.

1.3 Manfaat Praktikum

Manfaat dilakukannya praktikum ini adalah sebagai berikut :

1. Mahasiswa dapat memahami cara kerja algoritma sorting secara lebih jelas melalui proses pengurutan data yang dilakukan langkah demi langkah.
2. Mahasiswa memperoleh pengalaman dalam mengimplementasikan algoritma sorting ke dalam bahasa pemrograman Java, baik berbasis console maupun GUI.
3. Praktikum membantu meningkatkan kemampuan logika pemrograman dan pemahaman terhadap pengolahan data menggunakan array dan proses perulangan.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Sorting merupakan proses mengurutkan data berdasarkan aturan tertentu, seperti urutan angka dari kecil ke besar atau data teks secara alfabetis dari A-Z. Dalam pemrograman, sorting digunakan untuk mempermudah pencarian, pengelompokan, dan pengolahan data sehingga informasi menjadi lebih terstruktur dan mudah dibaca. Proses sorting banyak digunakan dalam berbagai aplikasi komputer, misalnya pengurutan data mahasiswa, nilai, produk, maupun data pada database. Semakin teratur susunan data, maka proses pengolahan data juga akan menjadi lebih cepat dan efisien.

1. Insertion Sort

Insertion Sort merupakan algoritma pengurutan yang bekerja dengan cara menyisipkan elemen ke posisi yang sesuai pada bagian array yang sudah terurut. Proses pengurutan dimulai dari elemen kedua, kemudian elemen tersebut dibandingkan dengan data sebelumnya. Jika nilai sebelumnya lebih besar, maka data akan digeser ke kanan sampai ditemukan posisi yang tepat untuk menyisipkan elemen baru. Metode ini dilakukan terus-menerus hingga seluruh data menjadi terurut dari nilai terkecil ke terbesar.

Pada program Insertion Sort, terdapat beberapa variabel penting seperti `arr_1016` yang digunakan untuk menyimpan data array, `n_1016` untuk menyimpan jumlah elemen array, `i_1016` sebagai penanda indeks perulangan utama, `key_1016` untuk menyimpan nilai sementara yang akan disisipkan, dan `j_1016` untuk membandingkan elemen sebelumnya. Program juga menggunakan perulangan `for` dan `while` untuk menjalankan proses penggeseran data sampai posisi yang sesuai ditemukan.

2. Selection Sort

Selection Sort adalah algoritma sorting yang bekerja dengan mencari nilai terkecil pada bagian array yang belum terurut, lalu menukarnya dengan elemen pada posisi awal. Setelah satu elemen terkecil dipindahkan, proses akan dilanjutkan ke bagian berikutnya sampai seluruh array berada dalam keadaan terurut.

Pada program Selection Sort terdapat variabel `arr_1016` sebagai tempat penyimpanan data array dan `n_1016` untuk menyimpan jumlah data. Variabel `i_1016` digunakan sebagai indeks utama proses sorting, sedangkan `j_1016` dipakai untuk mencari nilai terkecil pada bagian array yang belum terurut. Selain itu terdapat variabel `minIndex` untuk menyimpan posisi nilai terkecil sementara dan `temp_1016` sebagai variabel bantuan saat proses pertukaran data dilakukan.

3. Bubble Sort

Bubble Sort merupakan algoritma pengurutan yang dilakukan dengan cara membandingkan dua elemen yang berdekatan. Jika posisi kedua elemen belum sesuai, maka data akan ditukar. Proses ini dilakukan berulang kali hingga tidak ada lagi elemen yang perlu dipindahkan dan seluruh data menjadi terurut.

Pada program Bubble Sort digunakan variabel `arr_1016` untuk menyimpan array dan `n_1016` untuk menyimpan jumlah elemen array. Variabel `i_1016` digunakan untuk menghitung jumlah tahap pengurutan, sedangkan `j_1016` dipakai untuk membandingkan elemen yang berdekatan. Program juga menggunakan variabel `temp_1016` sebagai tempat penyimpanan sementara ketika dua data ditukar posisinya. Selain itu digunakan perulangan bertingkat `for` dan percabangan `if` untuk menentukan apakah data perlu ditukar atau tidak.

4. Insertion Sort GUI

Insertion Sort GUI merupakan pengembangan dari algoritma Insertion Sort yang ditampilkan dalam bentuk antarmuka grafis menggunakan Java Swing. Program ini memungkinkan pengguna memasukkan data array secara langsung, lalu melihat proses sorting berjalan langkah demi langkah melalui tampilan visual. Dengan adanya GUI, perpindahan data selama proses pengurutan menjadi lebih mudah diamati dibandingkan hanya menggunakan output console.

Pada program GUI terdapat beberapa komponen penting seperti JFrame sebagai jendela utama aplikasi, JPanel untuk wadah tampilan, JButton untuk tombol kontrol, JTextField untuk input array, JLabel untuk menampilkan elemen array, serta JTextArea untuk menampilkan log langkah sorting. Variabel `array_1016` digunakan untuk menyimpan data array, `labelArray_1016` untuk menyimpan label tampilan array, `stepButton_1016`, `resetButton_1016`, dan `setButton_1016` digunakan sebagai tombol kontrol program. Selain itu terdapat variabel `i_1016`, `j_1016`, `sorting_1016`, dan `stepCount_1016` yang berfungsi mengatur jalannya proses sorting langkah demi langkah pada GUI.

2.2 Kode Program

2.2.1 InsertionSort_2511531016

```
package pekan7_2511531016;

public class InsertionSort_2511531016 {
    public static void insertionSort_1016(int[] arr_1016) {
        int n_1016 = arr_1016.length;
        for (int i_1016 = 1; i_1016 < n_1016; i_1016++) {
            int key_1016 = arr_1016[i_1016];
            int j_1016 = i_1016 - 1;
            while (j_1016 >= 0 && arr_1016[j_1016] > key_1016) {
                arr_1016[j_1016 + 1] = arr_1016[j_1016];
                j_1016--;
            }
            arr_1016[j_1016 + 1] = key_1016;
        }
    }

    public static void main(String[] args) {
        int arr_1016[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_1016 = arr_1016.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i_1016 = 0; i_1016 < n_1016; i_1016++)
            System.out.print(arr_1016[i_1016] + " ");
        System.out.println("");
        insertionSort_1016(arr_1016);
        System.out.printf("array yang terurut:\n");
        for (int i_1016 = 0; i_1016 < n_1016; i_1016++)
            System.out.print(arr_1016[i_1016] + " ");
        System.out.println("");
    }
}
```

Kode Program 2.1 (Kode Program InsertionSort)

2.2.2 SelectionSort_2511531016

```
package pekan7_2511531016;

public class SelectionSort_2511531016 {
    public static void selectionSort_1016(int[] arr_1016) {
        int n_1016 = arr_1016.length;
        for (int i_1016 = 0; i_1016 < n_1016; i_1016++) {
            int minIndex = i_1016;
            for (int j_1016 = i_1016 + 1; j_1016 < n_1016; j_1016++) {
                if (arr_1016[j_1016] < arr_1016[minIndex]) {
                    minIndex = j_1016;
                }
            }
            int temp_1016 = arr_1016[i_1016];
            arr_1016[i_1016] = arr_1016[minIndex];
            arr_1016[minIndex] = temp_1016;
        }
    }

    public static void main(String[] args) {
        int arr_1016[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_1016 = arr_1016.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i_1016 = 0; i_1016 < n_1016; i_1016++)
            System.out.print(arr_1016[i_1016] + " ");
        System.out.println("");
        selectionSort_1016(arr_1016);
        System.out.printf("array yang terurut:\n");
        for (int i_1016 = 0; i_1016 < n_1016; i_1016++)
            System.out.print(arr_1016[i_1016] + " ");
        System.out.println("");
    }
}
```

Kode Program 2.2 (Kode Program SelectionSort)

2.2.3 BubbleSort_2511531016

```
package pekan7_2511531016;

public class BubbleSort_2511531016 {
    public static void bubbleSort_1016(int[] arr_1016) {
        int n_1016 = arr_1016.length;
        for (int i_1016 = 0; i_1016 < n_1016; i_1016++) {
            for (int j_1016 = 0; j_1016 < n_1016 - i_1016 - 1; j_1016++) {
                if (arr_1016[j_1016] > arr_1016[j_1016 + 1]) {
                    int temp_1016 = arr_1016[j_1016];
                    arr_1016[j_1016] = arr_1016[j_1016 + 1];
                    arr_1016[j_1016 + 1] = temp_1016;
                    // System.out.println("data:"+arr[j]+" "+arr[j+1]);
                }
            }
        }
    }

    public static void main(String[] args) {
        int arr_1016[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_1016 = arr_1016.length;
        System.out.print("array yang belum terurut: ");
        for (int i_1016 = 0; i_1016 < n_1016; i_1016++)
            System.out.print(arr_1016[i_1016] + " ");
        System.out.println("");
        bubbleSort_1016(arr_1016);
        System.out.print("array yang terurut menggunakan BubbleSort: ");
        for (int i_1016 = 0; i_1016 < n_1016; i_1016++)
            System.out.print(arr_1016[i_1016] + " ");
        System.out.println("");
    }
}
```

Kode Program 2.3 (Kode Program BubbleSort)

2.2.4 InsertionSortGUI_2511531016

```
package pekan7_2511531016;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JTextArea;
import javax.swing.border.EmptyBorder;

public class InsertionSortGUI_2511531016 extends JFrame {
    private static final long serialVersionUID = 1L;
    private JPanel contentPane_1016;
    private int[] array_1016;
    private JLabel[] labelArray_1016;
    private JButton stepButton_1016, resetButton_1016, setButton_1016;
    private JTextField inputField_1016;
    private JPanel panelArray_1016;
    private JTextArea stepArea_1016;
```

```

private int i_1016 = 1, j_1016;
private boolean sorting_1016 = false;
private int stepCount_1016 = 1;

/**
 * Create the frame.
 */
public InsertionSortGUI_2511531016() {
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setBounds(100, 100, 450, 300);
    contentPane_1016 = new JPanel();
    contentPane_1016.setBorder(new EmptyBorder(5, 5, 5, 5));
    setContentPane(contentPane_1016);

    setTitle("Insertion Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    // Panel input
    JPanel inputPanel_1016 = new JPanel(new FlowLayout());
    inputField_1016 = new JTextField(30);
    setButton_1016 = new JButton("Set Array");
    inputPanel_1016.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
    inputPanel_1016.add(inputField_1016);
    inputPanel_1016.add(setButton_1016);

    // Panel array visual
    panelArray_1016 = new JPanel();
    panelArray_1016.setLayout(new FlowLayout());

    // Panel kontrol
    JPanel controlPanel_1016 = new JPanel();
    stepButton_1016 = new JButton("Langkah Selanjutnya");
    resetButton_1016 = new JButton("Reset");
    stepButton_1016.setEnabled(false);
    controlPanel_1016.add(stepButton_1016);
    controlPanel_1016.add(resetButton_1016);

    // Area teks untuk log langkah-langkah
    stepArea_1016 = new JTextArea(8, 60);
    stepArea_1016.setEditable(false);
    stepArea_1016.setFont(new Font("Monospaced", Font.PLAIN, 14));
    JScrollPane scrollPane_1016 = new JScrollPane(stepArea_1016);

    // Tambahkan panel ke frame
    add(inputPanel_1016, BorderLayout.NORTH);
    add(panelArray_1016, BorderLayout.CENTER);
    add(controlPanel_1016, BorderLayout.SOUTH);
    add(scrollPane_1016, BorderLayout.EAST);

    // Event Set Array
    setButton_1016.addActionListener(e -> setArrayFromInput_1016());

    // Event Langkah Selanjutnya
    stepButton_1016.addActionListener(e -> performStep_1016());

    // Event Reset
    resetButton_1016.addActionListener(e -> reset_1016());
}

private void setArrayFromInput_1016() {
    String text_1016 = inputField_1016.getText().trim();
    if (text_1016.isEmpty()) return;
    String[] parts_1016 = text_1016.split(",");
    array_1016 = new int[parts_1016.length];
    try {
        for (int k_1016 = 0; k_1016 < parts_1016.length; k_1016++) {
            array_1016[k_1016] = Integer.parseInt(parts_1016[k_1016].trim());
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan " + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    i_1016 = 1;
    sorting_1016 = true;
    stepButton_1016.setEnabled(true);
    stepArea_1016.setText("");
    panelArray_1016.removeAll();
    labelArray_1016 = new JLabel[array_1016.length];
}

```



```

        for (int k_1016 = 0; k_1016 < array_1016.length; k_1016++) {
            labelArray_1016[k_1016] = new JLabel(String.valueOf(array_1016[k_1016]));
            labelArray_1016[k_1016].setFont(new Font("Arial", Font.BOLD, 24));
            labelArray_1016[k_1016].setBorder(BorderFactory.createLineBorder(Color.BLACK));
            labelArray_1016[k_1016].setPreferredSize(new Dimension(50, 50));
            labelArray_1016[k_1016].setHorizontalAlignment(SwingConstants.CENTER);
            panelArray_1016.add(labelArray_1016[k_1016]);
        }
        panelArray_1016.revalidate();
        panelArray_1016.repaint();
    }

    private void performStep_1016() {
        if (i_1016 < array_1016.length && sorting_1016) {
            int key_1016 = array_1016[i_1016];
            j_1016 = i_1016 - 1;
            StringBuilder stepLog_1016 = new StringBuilder();
            stepLog_1016.append("Langkah ").append(stepCount_1016).
                append(": Memasukkan ").append(key_1016).append("\n");
            while (j_1016 >= 0 && array_1016[j_1016] > key_1016) {
                array_1016[j_1016 + 1] = array_1016[j_1016];
                j_1016--;
            }
            array_1016[j_1016 + 1] = key_1016;
            updateLabels_1016();
            stepLog_1016.append("Hasil: ").append(arrayToString_1016(array_1016)).append("\n\n");
            stepArea_1016.append(stepLog_1016.toString());
            i_1016++;
            stepCount_1016++;
            if (i_1016 == array_1016.length) {
                sorting_1016 = false;
                stepButton_1016.setEnabled(false);
                JOptionPane.showMessageDialog(this, "Sorting selesai!");
            }
        }

        private void updateLabels_1016() {
            for (int k_1016 = 0; k_1016 < array_1016.length; k_1016++) {
                labelArray_1016[k_1016].setText(String.valueOf(array_1016[k_1016]));
            }
        }

        private void reset_1016() {
            inputField_1016.setText("");
            panelArray_1016.removeAll();
            panelArray_1016.revalidate();
            panelArray_1016.repaint();
            stepArea_1016.setText("");
            stepButton_1016.setEnabled(false);
            sorting_1016 = false;
            i_1016 = 1;
            stepCount_1016 = 1;
        }

        private String arrayToString_1016(int[] arr_1016) {
            StringBuilder sb_1016 = new StringBuilder();
            for (int k_1016 = 0; k_1016 < arr_1016.length; k_1016++) {
                sb_1016.append(arr_1016[k_1016]);
                if (k_1016 < arr_1016.length - 1) sb_1016.append(", ");
            }
            return sb_1016.toString();
        }

        public static void main(String[] args) {
            SwingUtilities.invokeLater(() -> {
                InsertionSortGUI_2511531016 gui_1016 = new InsertionSortGUI_2511531016();
                gui_1016.setVisible(true);
            });
        }
    }

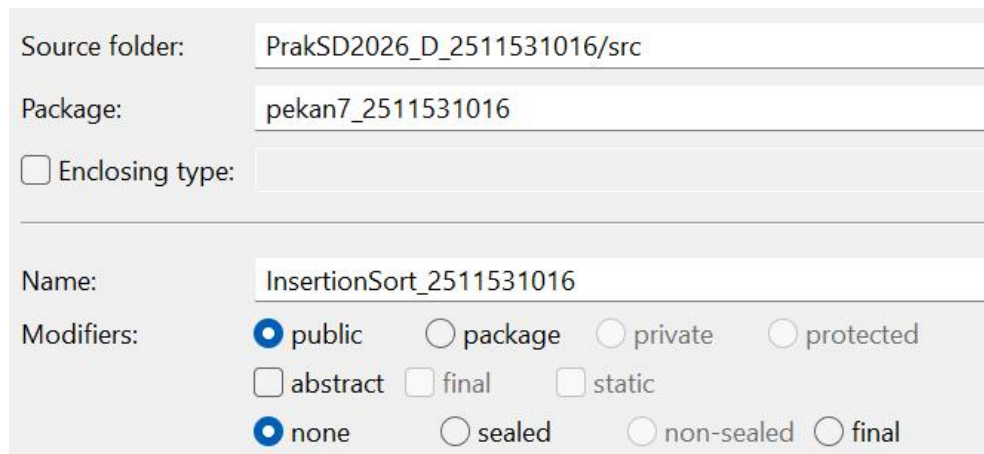
```

Kode Program 2.4 (Kode Program InsertionSortGUI)

2.3 Langkah Kerja

2.3.1 InsertionSort_2511531016

1. Membuat kelas baru bernama *InsertionSort_2511531016* dalam *package* *pekan7_2511531016*.



Source folder: PrakSD2026_D_2511531016/src

Package: pekan7_2511531016

☐ Enclosing type:

Name: InsertionSort_2511531016

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Gambar 2.1 (Langkah 1 InsertionSort)

2. Program diawali dengan deklarasi package dan pembuatan class. Lalu, buat method `insertionSort_1016()` yang digunakan untuk melakukan proses pengurutan data menggunakan metode Insertion Sort. Variabel `n_1016` dipakai untuk menyimpan jumlah elemen array yang akan diurutkan.

```
package pekan7_2511531016;  
  
public class InsertionSort_2511531016 {  
    public static void insertionSort_1016(int[] arr_1016) {  
        int n_1016 = arr_1016.length;  
    }  
}
```

Gambar 2.2 (Langkah 2 InsertionSort)

3. Kemudian, lakukan perulangan for mulai dari indeks ke-1 untuk memeriksa setiap elemen array. Nilai array pada indeks tersebut disimpan ke dalam variabel `key_1016`, sedangkan variabel `j_1016` digunakan untuk

menunjuk elemen sebelumnya yang akan dibandingkan selama proses pengurutan berlangsung.

```
for (int i_1016 = 1; i_1016 < n_1016; i_1016++) {  
    int key_1016 = arr_1016[i_1016];  
    int j_1016 = i_1016 - 1;
```

Gambar 2.3 (Langkah 3 InsertionSort)

4. Jalankan perulangan while untuk membandingkan nilai key_1016 dengan elemen-elemen sebelumnya. Jika elemen sebelumnya lebih besar, maka elemen tersebut digeser satu posisi ke kanan. Setelah posisi yang sesuai ditemukan, nilai key_1016 dimasukkan ke tempat yang benar sehingga bagian array sebelah kiri tetap dalam keadaan terurut.

```
while (j_1016 >= 0 && arr_1016[j_1016] > key_1016) {  
    arr_1016[j_1016 + 1] = arr_1016[j_1016];  
    j_1016--;  
}  
arr_1016[j_1016 + 1] = key_1016;
```

Gambar 2.4 (Langkah 4 InsertionSort)

5. Setelah method sorting dibuat, masuk ke method main(). Pada bagian ini array arr_1016 diinisialisasi dengan beberapa data acak, lalu variabel n_1016 digunakan untuk menyimpan jumlah elemen array. tampilkan juga teks sebagai penanda bahwa data yang dicetak merupakan array sebelum diurutkan.

```
public static void main(String[] args) {  
    int arr_1016[] = { 23, 78, 45, 8, 32, 56, 1 };  
    int n_1016 = arr_1016.length;  
    System.out.printf("array yang belum terurut:\n");
```

Gambar 2.5 (Langkah 5 InsertionSort)

6. Gunakan perulangan for untuk menampilkan seluruh isi array sebelum proses sorting dilakukan. Setiap elemen dicetak satu per satu agar pengguna dapat melihat urutan data awal yang masih belum teratur.

```
for (int i_1016 = 0; i_1016 < n_1016; i_1016++)  
System.out.print(arr_1016[i_1016] + " ");  
System.out.println("");  
insertionSort_1016(arr_1016);  
System.out.printf("array yang teratur:\n");
```

Gambar 2.6 (Langkah 6 InsertionSort)

7. Terakhir, panggil method `insertionSort_1016(arr_1016)` untuk mengurutkan data. Setelah proses selesai, gunakan kembali perulangan for untuk menampilkan isi array yang sudah terurut dari nilai terkecil ke terbesar.

```
for (int i_1016 = 0; i_1016 < n_1016; i_1016++)  
System.out.print(arr_1016[i_1016] + " ");  
System.out.println("");
```

Gambar 2.7 (Langkah 7 InsertionSort)

2.3.2 SelectionSort_2511531016

1. Membuat kelas baru bernama *SelectionSort_2511531016* dalam package *pekan7_2511531016*.

Source folder:

Package:

☐ Enclosing type:

Name:

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Gambar 2.8 (Langkah 1 SelectionSort)

2. Deklarasikan package dan pembuatan class. Lalu, buat method selectionSort_1016() yang digunakan untuk melakukan proses pengurutan data menggunakan metode Selection Sort. Variabel n_1016 digunakan untuk menyimpan jumlah elemen array yang akan diproses.

```
package pekan7_2511531016;

public class SelectionSort_2511531016 {
    public static void selectionSort_1016(int[] arr_1016) {
        int n_1016 = arr_1016.length;
```

Gambar 2.9 (Langkah 2 SelectionSort)

3. Kemudian, jalankan perulangan for untuk mencari nilai terkecil pada array. Variabel minIndex digunakan untuk menyimpan indeks elemen terkecil sementara. Lalu perulangan kedua digunakan untuk membandingkan setiap elemen setelah indeks saat ini dengan nilai pada minIndex. Jika ditemukan nilai yang lebih kecil, maka minIndex akan diperbarui.

```
for (int i_1016 = 0; i_1016 < n_1016; i_1016++) {
    int minIndex = i_1016;
    for (int j_1016 = i_1016 + 1; j_1016 < n_1016; j_1016++) {
        if (arr_1016[j_1016] < arr_1016[minIndex]) {
            minIndex = j_1016;
```

Gambar 2.10 (Langkah 3 SelectionSort)

4. Setelah elemen terkecil ditemukan, lakukan proses pertukaran data menggunakan variabel sementara temp_1016. Nilai pada indeks awal ditukar dengan nilai terkecil yang telah ditemukan sehingga posisi elemen menjadi lebih terurut secara bertahap.

```
int temp_1016 = arr_1016[i_1016];
arr_1016[i_1016] = arr_1016[minIndex];
arr_1016[minIndex] = temp_1016;
```

Gambar 2.11 (Langkah 4 SelectionSort)

5. Setelah method sorting selesai dibuat, masuk ke method main(). Pada bagian ini array arr_1016 diisi dengan beberapa data acak yang nantinya akan diurutkan. Variabel n_1016 digunakan untuk menyimpan jumlah elemen array, kemudian tampilkan teks penanda sebelum data awal dicetak.

```
public static void main(String[] args) {  
    int arr_1016[] = { 23, 78, 45, 8, 32, 56, 1 };  
    int n_1016 = arr_1016.length;  
    System.out.printf("array yang belum terurut:\n");
```

Gambar 2.12 (Langkah 5 SelectionSort)

6. Gunakan perulangan for untuk menampilkan seluruh isi array sebelum proses pengurutan dilakukan. Setiap elemen dicetak satu per satu agar pengguna dapat melihat kondisi data yang masih belum terurut.

```
for (int i_1016 = 0; i_1016 < n_1016; i_1016++)  
    System.out.print(arr_1016[i_1016] + " ");  
System.out.println("");  
selectionSort_1016(arr_1016);  
System.out.printf("array yang terurut:\n");
```

Gambar 2.13 (Langkah 6 SelectionSort)

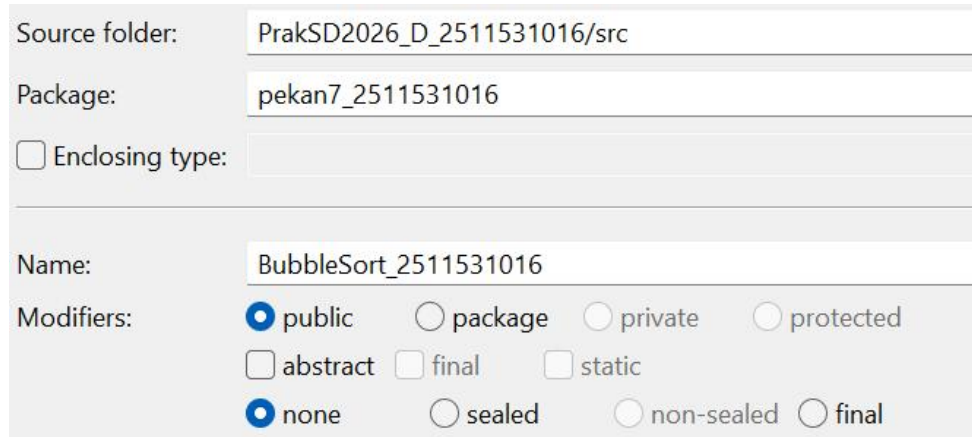
7. Terakhir, panggil method selectionSort_1016(arr_1016) untuk mengurutkan data menggunakan metode Selection Sort. Setelah proses selesai, tampilkan kembali isi array menggunakan perulangan for sehingga hasil pengurutan dari nilai terkecil ke terbesar dapat terlihat pada output.

```
for (int i_1016 = 0; i_1016 < n_1016; i_1016++)  
    System.out.print(arr_1016[i_1016] + " ");  
System.out.println("");
```

Gambar 2.14 (Langkah 7 SelectionSort)

2.3.3 BubbleSort_2511531016

1. Membuat kelas baru bernama *BubbleSort_2511531016* dalam *package pekan7_2511531016*.



Source folder:	PrakSD2026_D_2511531016/src
Package:	pekan7_2511531016
☐ Enclosing type:	
Name:	BubbleSort_2511531016
Modifiers:	☒ public ☐ package ☐ private ☐ protected ☐ abstract ☐ final ☐ static ☒ none ☐ sealed ☐ non-sealed ☐ final

Gambar 2.15 (Langkah 1 BubbleSort)

2. Program dimulai dengan deklarasi package dan pembuatan class. Lalu, buat method `bubbleSort_1016()` yang digunakan untuk mengurutkan data menggunakan metode Bubble Sort. Variabel `n_1016` dipakai untuk menyimpan jumlah elemen array yang akan diproses selama pengurutan.

```
package pekan7_2511531016;  
  
public class BubbleSort_2511531016 {  
    public static void bubbleSort_1016(int[] arr_1016) {  
        int n_1016 = arr_1016.length;  
    }  
}
```

Gambar 2.16 (Langkah 2 BubbleSort)

3. Kemudian, jalankan perulangan for bertingkat untuk melakukan proses pengurutan. Perulangan luar digunakan untuk menentukan jumlah tahap pengurutan, sedangkan perulangan dalam digunakan untuk membandingkan elemen array yang bersebelahan dari awal hingga batas tertentu.

```
for (int i_1016 = 0; i_1016 < n_1016; i_1016++) {
```

```
for (int j_1016 = 0; j_1016 < n_1016 - i_1016 - 1; j_1016++) {
```

Gambar 2.17 (Langkah 3 BubbleSort)

4. Pada bagian ini lakukan pengecekan kondisi menggunakan if. Jika nilai elemen sebelah kiri lebih besar daripada elemen di sebelah kanan, maka kedua nilai tersebut akan ditukar menggunakan variabel sementara temp_1016. Proses pertukaran ini dilakukan terus-menerus hingga data terbesar berpindah ke bagian akhir array pada setiap tahap pengurutan.

```
if (arr_1016[j_1016] > arr_1016[j_1016 + 1]) {
    int temp_1016 = arr_1016[j_1016];
    arr_1016[j_1016] = arr_1016[j_1016 + 1];
    arr_1016[j_1016 + 1] = temp_1016;
    // System.out.println("data:"+arr[j]+" "+arr[j+1]);
}
```

Gambar 2.18 (Langkah 4 BubbleSort)

5. Setelah method sorting selesai dibuat, masuk ke method main(). Pada bagian ini array arr_1016 diisi dengan beberapa data acak yang nantinya akan diurutkan. Variabel n_1016 digunakan untuk menyimpan jumlah elemen array, kemudian tampilkan teks sebagai penanda sebelum data awal dicetak.

```
public static void main(String[] args) {
    int arr_1016[] = { 23, 78, 45, 8, 32, 56, 1 };
    int n_1016 = arr_1016.length;
    System.out.print("array yang belum terurut: ");
}
```

Gambar 2.19 (Langkah 5 BubbleSort)

6. Gunakan perulangan for untuk menampilkan seluruh isi array sebelum proses pengurutan dilakukan. Setiap elemen array dicetak satu per satu sehingga pengguna dapat melihat urutan data yang masih belum teratur.

```
for (int i_1016 = 0; i_1016 < n_1016; i_1016++)
    System.out.print(arr_1016[i_1016] + " ");
System.out.println("");
```

```

bubbleSort_1016(arr_1016);
System.out.print("array yang terurut menggunakan BubbleSort: ");

```

Gambar 2.20 (Langkah 6 BubbleSort)

7. Terakhir, panggil method bubbleSort_1016(arr_1016) untuk mengurutkan data menggunakan metode Bubble Sort. Setelah proses selesai, gunakan kembali perulangan for untuk menampilkan isi array yang sudah terurut dari nilai terkecil ke terbesar.

```

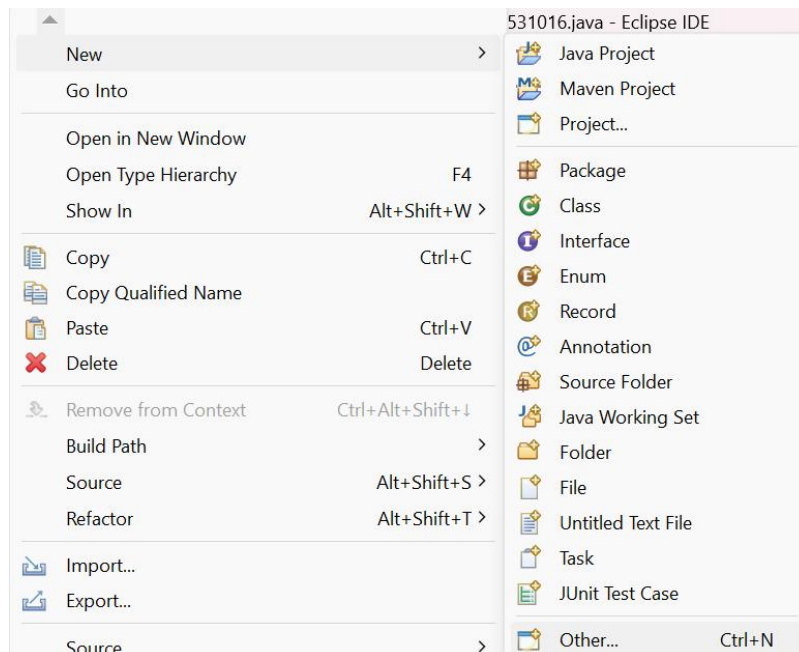
for (int i_1016 = 0; i_1016 < n_1016; i_1016++)
System.out.print(arr_1016[i_1016] + " ");
System.out.println("");

```

Gambar 2.21 (Langkah 7 BubbleSort)

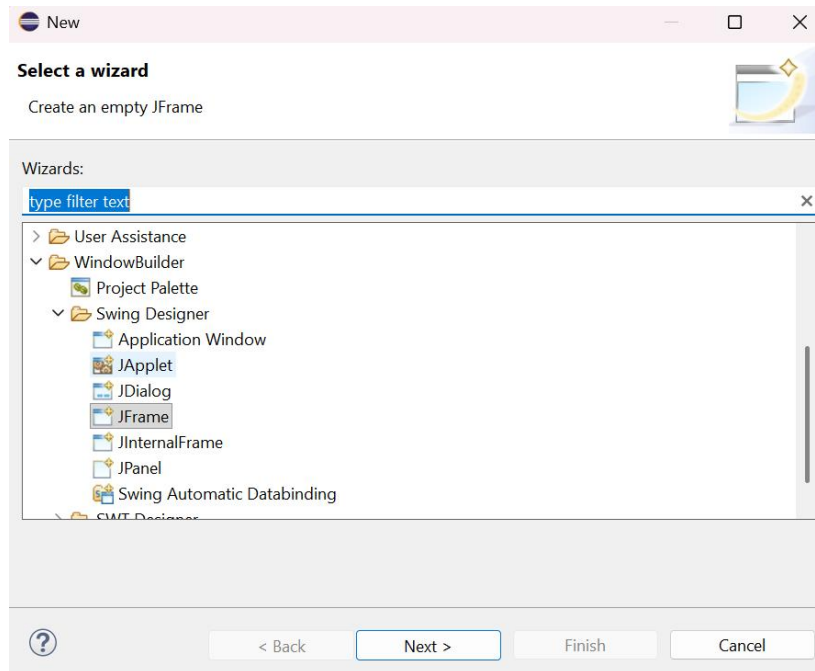
2.3.4 InsertionSortGUI_2511531016

1. Untuk membuat class GUI, klik kanan pada package pekan7_2511531016, lalu pilih “New”, pilih “Other”.



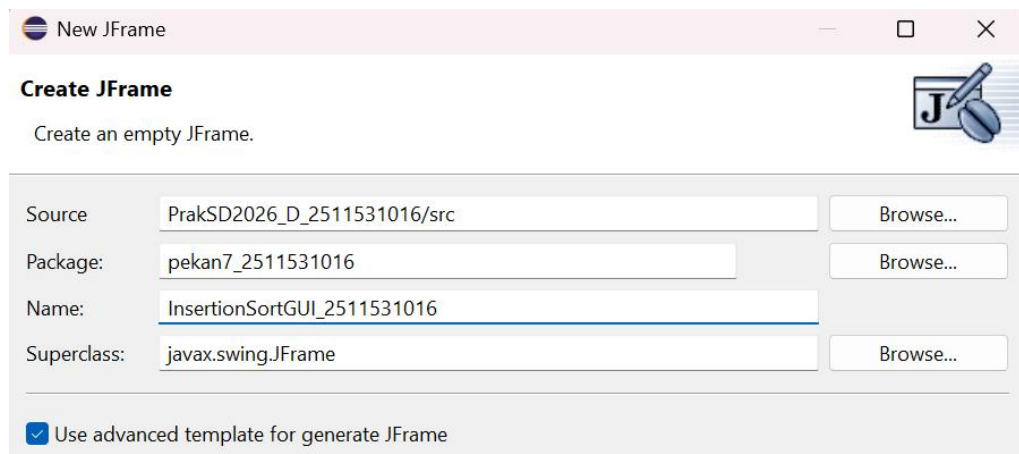
Gambar 2.22 (Langkah 1 InsertionSortGUI)

2. Selanjutnya, klik WindowBuilder dan pilih JFrame. Klik Next.



Gambar 2.23 (Langkah 2 InsertionSortGUI)

3. Kemudian, isikan nama class yaitu InsertionSortGUI_2511531016. Klik Finish.



Gambar 2.24 (Langkah 3 InsertionSortGUI)

4. Program diawali dengan deklarasi package serta import beberapa library Java Swing dan AWT yang digunakan untuk membuat tampilan GUI. Library tersebut dipakai untuk membuat frame, panel, tombol, text field, label, area teks, hingga pengaturan warna dan font pada aplikasi visualisasi Insertion Sort.

```
package pekan7_2511531016;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JTextArea;
import javax.swing.border.EmptyBorder;
```

Gambar 2.25 (Langkah 4 InsertionSortGUI)

5. Kemudian, deklarasikan class InsertionSortGUI_2511531016 yang merupakan turunan dari JFrame. Pada bagian ini juga buat beberapa variabel seperti array, label array, tombol, text field, panel, dan text area yang digunakan untuk menampilkan proses pengurutan secara visual pada GUI.

```
public class InsertionSortGUI_2511531016 extends JFrame {
    private static final long serialVersionUID = 1L;
    private JPanel contentPane_1016;
    private int[] array_1016;
    private JLabel[] labelArray_1016;
    private JButton stepButton_1016, resetButton_1016, setButton_1016;
    private JTextField inputField_1016;
    private JPanel panelArray_1016;
    private JTextArea stepArea_1016;
    private int i_1016 = 1, j_1016;
```



```
private boolean sorting_1016 = false;
private int stepCount_1016 = 1;
```

Gambar 2.26 (Langkah 5 InsertionSortGUI)

6. Buat constructor InsertionSortGUI_2511531016() untuk membangun tampilan utama program. Pada bagian ini diatur ukuran frame, judul aplikasi, posisi tampilan, layout, serta panel utama yang akan menampung seluruh komponen GUI.

```
/**
 * Create the frame.
 */
public InsertionSortGUI_2511531016() {
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setBounds(100, 100, 450, 300);
    contentPane_1016 = new JPanel();
    contentPane_1016.setBorder(new EmptyBorder(5, 5, 5, 5));
    setContentPane(contentPane_1016);
    setTitle("Insertion Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());
```

Gambar 2.27 (Langkah 6 InsertionSortGUI)

7. Lalu, buat panel input menggunakan JPanel dengan layout FlowLayout. Pada panel ini terdapat JTextField untuk memasukkan data array dan tombol Set Array yang digunakan untuk memproses input angka dari pengguna.

```
// Panel input
JPanel inputPanel_1016 = new JPanel(new FlowLayout());
inputField_1016 = new JTextField(30);
setButton_1016 = new JButton("Set Array");
inputPanel_1016.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
inputPanel_1016.add(inputField_1016);
inputPanel_1016.add(setButton_1016);
```

Gambar 2.28 (Langkah 7 InsertionSortGUI)

8. Setelah itu dibuat panel visual array bernama `panelArray_1016`. Panel ini digunakan untuk menampilkan isi array dalam bentuk label sehingga proses pengurutan dapat dilihat secara visual langkah demi langkah.

```
// Panel array visual
panelArray_1016 = new JPanel();
panelArray_1016.setLayout(new FlowLayout());
```

Gambar 2.29 (Langkah 8 InsertionSortGUI)

9. Buat panel kontrol yang berisi tombol Langkah Selanjutnya dan Reset. Tombol langkah digunakan untuk menjalankan proses sorting satu per satu, sedangkan tombol reset dipakai untuk mengembalikan tampilan program ke kondisi awal.

```
// Panel kontrol
JPanel controlPanel_1016 = new JPanel();
stepButton_1016 = new JButton("Langkah Selanjutnya");
resetButton_1016 = new JButton("Reset");
stepButton_1016.setEnabled(false);
controlPanel_1016.add(stepButton_1016);
controlPanel_1016.add(resetButton_1016);
```

Gambar 2.30 (Langkah 9 InsertionSortGUI)

10. Selanjutnya, buat `JTextArea` bernama `stepArea_1016` yang digunakan untuk menampilkan log atau penjelasan setiap langkah proses sorting. Area teks ini dimasukkan ke dalam `JScrollPane` agar pengguna dapat melihat seluruh riwayat langkah meskipun teks sudah panjang.

```
// Area teks untuk log langkah-langkah
stepArea_1016 = new JTextArea(8, 60);
stepArea_1016.setEditable(false);
stepArea_1016.setFont(new Font("Monospaced", Font.PLAIN, 14));
JScrollPane scrollPane_1016 = new JScrollPane(stepArea_1016);
```

Gambar 2.31 (Langkah 10 InsertionSortGUI)

11. Setelah semua komponen dibuat, tambahkan panel-panel tersebut ke dalam frame menggunakan BorderLayout. Panel input ditempatkan di bagian atas, panel array di tengah, panel kontrol di bawah, dan area log di sisi kanan tampilan.

```
// Tambahkan panel ke frame
add(inputPanel_1016, BorderLayout.NORTH);
add(panelArray_1016, BorderLayout.CENTER);
add(controlPanel_1016, BorderLayout.SOUTH);
add(scrollPane_1016, BorderLayout.EAST);
```

Gambar 2.32 (Langkah 11 InsertionSortGUI)

12. Tambahkan event listener pada setiap tombol. Tombol Set Array digunakan untuk membaca input array, tombol Langkah Selanjutnya dipakai untuk menjalankan satu langkah sorting, sedangkan tombol Reset digunakan untuk menghapus data dan mengulang program dari awal.

```
// Event Set Array
setButton_1016.addActionListener(e -> setArrayFromInput_1016());

// Event Langkah Selanjutnya
stepButton_1016.addActionListener(e -> performStep_1016());

// Event Reset
resetButton_1016.addActionListener(e -> reset_1016());
```

Gambar 2.33 (Langkah 12 InsertionSortGUI)

13. Pada method setArrayFromInput_1016(), program membaca input angka dari pengguna yang dipisahkan dengan tanda koma. Data input tersebut dipecah menggunakan split(","), lalu dikonversi menjadi tipe integer dan disimpan ke dalam array.

```
private void setArrayFromInput_1016() {
    String text_1016 = inputField_1016.getText().trim();
    if (text_1016.isEmpty()) return;
    String[] parts_1016 = text_1016.split(",");
    array_1016 = new int[parts_1016.length];
    try {
```

```

for (int k_1016 = 0; k_1016 < parts_1016.length; k_1016++) {
array_1016[k_1016] = Integer.parseInt(parts_1016[k_1016].trim());
}
} catch (NumberFormatException e) {
JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang
dipisahkan " + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
return;
}

```

Gambar 2.34 (Langkah 13 InsertionSortGUI)

14. Setelah array berhasil dibuat, inisialisasi ulang variabel sorting dan bersihkan tampilan sebelumnya. Kemudian setiap elemen array tampilkan dalam bentuk JLabel dengan ukuran, font, dan border tertentu agar proses sorting dapat terlihat lebih jelas secara visual.

```

i_1016 = 1;
sorting_1016 = true;
stepButton_1016.setEnabled(true);
stepArea_1016.setText("");
panelArray_1016.removeAll();
labelArray_1016 = new JLabel[array_1016.length];
for (int k_1016 = 0; k_1016 < array_1016.length; k_1016++) {
labelArray_1016[k_1016] = new
JLabel(String.valueOf(array_1016[k_1016]));
labelArray_1016[k_1016].setFont(new Font("Arial", Font.BOLD, 24));
labelArray_1016[k_1016].setBorder(BorderFactory.createLineBorder(Color.BLACK));
labelArray_1016[k_1016].setPreferredSize(new Dimension(50, 50));
labelArray_1016[k_1016].setHorizontalAlignment(SwingConstants.CENTER);
;
panelArray_1016.add(labelArray_1016[k_1016]);
}
panelArray_1016.revalidate();
panelArray_1016.repaint();

```

Gambar 2.35 (Langkah 14 InsertionSortGUI)

15. Pada method performStep_1016(), jalankan proses Insertion Sort langkah demi langkah. Variabel key_1016 digunakan untuk menyimpan nilai yang akan disisipkan, sedangkan j_1016 digunakan untuk membandingkan elemen sebelumnya selama proses penggeseran data berlangsung.

```

private void performStep_1016() {
    if (i_1016 < array_1016.length && sorting_1016) {
        int key_1016 = array_1016[i_1016];
        j_1016 = i_1016 - 1;
        StringBuilder stepLog_1016 = new StringBuilder();
        stepLog_1016.append("Langkah ").append(stepCount_1016).
            append(": Memasukkan ").append(key_1016).append("\n");
        while (j_1016 >= 0 && array_1016[j_1016] > key_1016) {
            array_1016[j_1016 + 1] = array_1016[j_1016];
            j_1016--;
        }
    }
}

```

Gambar 2.36 (Langkah 15 InsertionSortGUI)

16. Setelah posisi yang tepat ditemukan, nilai key_1016 dimasukkan kembali ke array. kemudian perbarui tampilan label array dan tambahkan hasil langkah sorting ke dalam stepArea_1016 sehingga pengguna dapat melihat perubahan array pada setiap proses.

```

array_1016[j_1016 + 1] = key_1016;
updateLabels_1016();
stepLog_1016.append("Hasil:
").append(arrayToString_1016(array_1016)).append("\n\n");
stepArea_1016.append(stepLog_1016.toString());
i_1016++;
stepCount_1016++;
if (i_1016 == array_1016.length) {
    sorting_1016 = false;
    stepButton_1016.setEnabled(false);
    JOptionPane.showMessageDialog(this, "Sorting selesai!");
}

```

Gambar 2.37 (Langkah 16 InsertionSortGUI)

17. Method updateLabels_1016() digunakan untuk memperbarui isi setiap label sesuai data terbaru dalam array. Dengan cara ini tampilan GUI akan selalu menyesuaikan hasil sorting terbaru setelah setiap langkah dijalankan.

```

private void updateLabels_1016() {
    for (int k_1016 = 0; k_1016 < array_1016.length; k_1016++) {
        labelArray_1016[k_1016].setText(String.valueOf(array_1016[k_1016]));
    }
}

```

Gambar 2.38 (Langkah 17 InsertionSortGUI)

18. Pada method `reset_1016()`, hapus seluruh input, bersihkan panel array dan area log, lalu kembalikan variabel sorting ke kondisi awal. Tombol langkah juga dinonaktifkan kembali sampai pengguna memasukkan data baru.

```
private void reset_1016() {
    inputField_1016.setText("");
    panelArray_1016.removeAll();
    panelArray_1016.revalidate();
    panelArray_1016.repaint();
    stepArea_1016.setText("");
    stepButton_1016.setEnabled(false);
    sorting_1016 = false;
    i_1016 = 1;
    stepCount_1016 = 1;
}
```

Gambar 2.39 (Langkah 18 InsertionSortGUI)

19. Method `arrayToString_1016()` digunakan untuk mengubah isi array menjadi bentuk string. Setiap elemen array digabungkan dengan tanda koma sehingga hasil array lebih mudah ditampilkan pada area log langkah-langkah sorting.

```
private String arrayToString_1016(int[] arr_1016) {
    StringBuilder sb_1016 = new StringBuilder();
    for (int k_1016 = 0; k_1016 < arr_1016.length; k_1016++) {
        sb_1016.append(arr_1016[k_1016]);
        if (k_1016 < arr_1016.length - 1) sb_1016.append(", ");
    }
    return sb_1016.toString();
}
```

Gambar 2.40 (Langkah 18 InsertionSortGUI)

20. Terakhir, pada method `main()`, jalankan GUI menggunakan `SwingUtilities.invokeLater()`. Setelah objek `InsertionSortGUI_2511531016` dibuat, tampilan aplikasi akan muncul dan siap digunakan oleh pengguna untuk melihat proses Insertion Sort secara visual.

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        InsertionSortGUI_2511531016 gui_1016 = new
        InsertionSortGUI_2511531016();
    });
}
```

```
gui_1016.setVisible(true);  
});
```

Gambar 2.41 (Langkah 18 InsertionSortGUI)

2.4 Analisis Hasil

Berdasarkan hasil praktikum, program sorting yang dibuat menggunakan metode Insertion Sort, Selection Sort, dan Bubble Sort berhasil berjalan sesuai tujuan. Ketiga program mampu mengurutkan data dari kondisi awal yang masih acak menjadi urutan ascending dengan hasil akhir 1 8 23 32 45 56 78. Dari proses yang diamati, setiap algoritma memiliki cara kerja yang berbeda walaupun hasil akhirnya sama. Insertion Sort bekerja dengan menyisipkan data ke posisi yang tepat pada bagian array yang sudah terurut, Selection Sort memilih nilai terkecil pada setiap tahap lalu menukarnya ke posisi awal, sedangkan Bubble Sort melakukan pertukaran antar elemen yang berdekatan secara berulang. Perbedaan proses ini membuat praktikum lebih mudah dipahami karena setiap metode memiliki alur pengurutan yang khas.

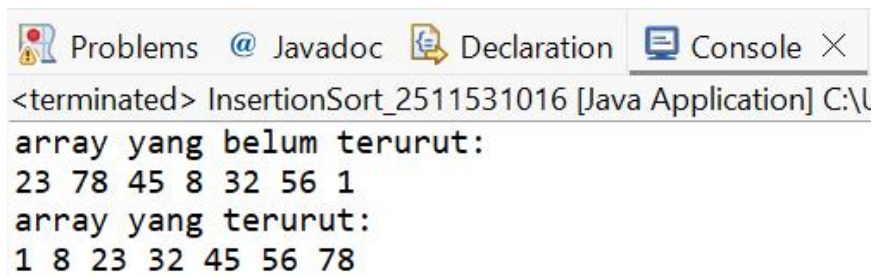
Selain program berbasis console, praktikum juga menampilkan visualisasi GUI pada Insertion Sort sehingga proses pengurutan dapat diamati langkah demi langkah. Tampilan visual ini membantu memahami bagaimana data berpindah posisi selama sorting berlangsung, terutama saat elemen dibandingkan dan disisipkan ke tempat yang sesuai. Dari hasil percobaan dapat dilihat bahwa algoritma sorting tidak hanya menghasilkan data terurut, tetapi juga memperlihatkan logika proses pengurutan secara bertahap. Dengan adanya tampilan langkah-langkah pada GUI, proses belajar terasa lebih jelas dibanding hanya melihat hasil akhir array saja.

BAB III

KESIMPULAN

3.1 Hasil Praktikum

3.1.1 InsertionSort_2511531016

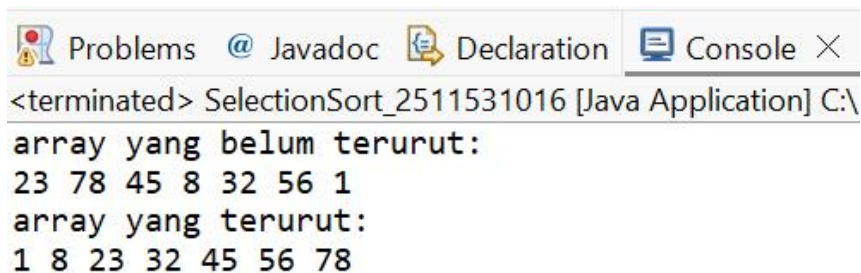


```
<terminated> InsertionSort_2511531016 [Java Application] C:\narray yang belum terurut:\n23 78 45 8 32 56 1\narray yang terurut:\n1 8 23 32 45 56 78
```

Gambar 3.1 (Hasil Praktikum InsertionSort)

Output program menampilkan kondisi array sebelum dan sesudah dilakukan proses pengurutan menggunakan algoritma Insertion Sort. Pada awalnya data masih dalam keadaan acak yaitu 23 78 45 8 32 56 1. Setelah proses sorting dijalankan, setiap elemen dibandingkan dan disisipkan ke posisi yang sesuai sehingga array berubah menjadi urut secara ascending menjadi 1 8 23 32 45 56 78. Hasil tersebut menunjukkan bahwa metode Insertion Sort berhasil mengurutkan data dengan cara menyisipkan elemen satu per satu ke bagian array yang sudah terurut sebelumnya.

3.1.2 SelectionSort_2511531016

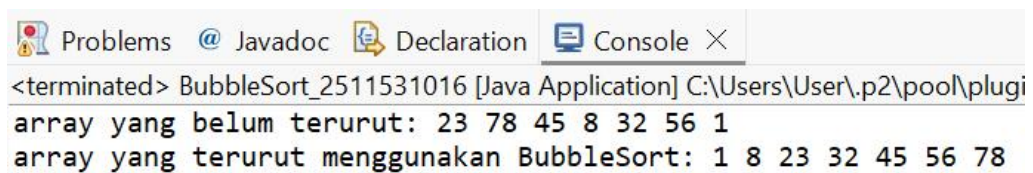


```
<terminated> SelectionSort_2511531016 [Java Application] C:\narray yang belum terurut:\n23 78 45 8 32 56 1\narray yang terurut:\n1 8 23 32 45 56 78
```

Gambar 3.2 (Hasil Praktikum SelectionSort)

Output program menampilkan kondisi array sebelum dan sesudah dilakukan proses pengurutan menggunakan algoritma Selection Sort. Pada awalnya data masih dalam keadaan acak yaitu 23 78 45 8 32 56 1. Selama proses sorting, program akan mencari nilai terkecil pada array lalu menukarnya ke posisi yang sesuai secara bertahap. Setelah seluruh proses selesai, array berhasil terurut secara ascending menjadi 1 8 23 32 45 56 78. Hasil tersebut menunjukkan bahwa metode Selection Sort bekerja dengan memilih elemen terkecil pada setiap tahap hingga semua data tersusun dari nilai terkecil ke terbesar.

3.1.3 BubbleSort_2511531016

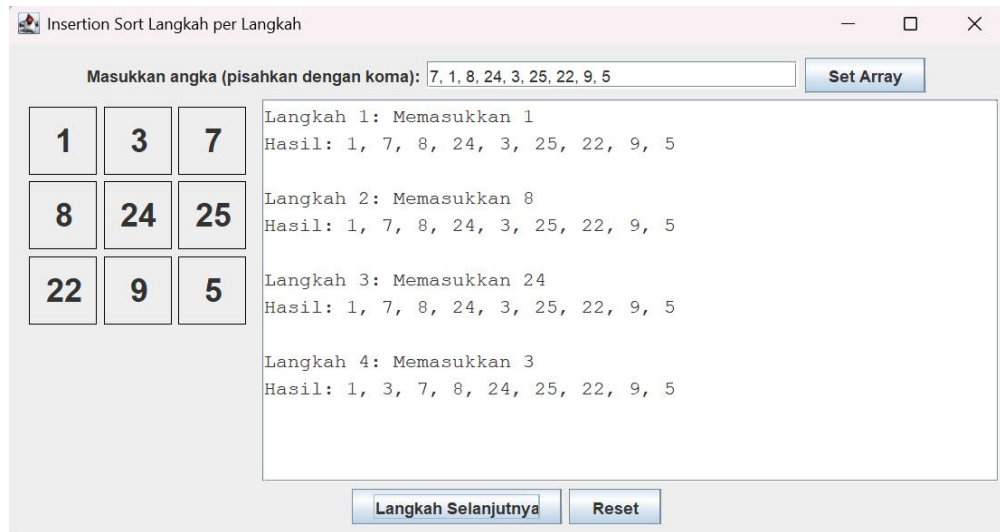


```
<terminated> BubbleSort_2511531016 [Java Application] C:\Users\User\p2\pool\plugi  
array yang belum terurut: 23 78 45 8 32 56 1  
array yang terurut menggunakan BubbleSort: 1 8 23 32 45 56 78
```

Gambar 3.3 (Hasil Praktikum BubbleSort)

Output program menampilkan kondisi array sebelum dan sesudah dilakukan proses pengurutan menggunakan algoritma Bubble Sort. Pada awalnya data masih dalam keadaan acak yaitu 23 78 45 8 32 56 1. Selama proses sorting, setiap elemen dibandingkan dengan elemen di sebelahnya, lalu ditukar jika urutannya belum sesuai. Proses ini dilakukan berulang kali hingga nilai terbesar berpindah ke bagian akhir array pada setiap tahap pengurutan. Setelah seluruh proses selesai, array berhasil terurut secara ascending menjadi 1 8 23 32 45 56 78, yang menandakan bahwa metode Bubble Sort berjalan dengan baik.

3.1.4 InsertionSortGUI_2511531016



Gambar 3.4 (Hasil Praktikum InsertionSortGUI)

Output program di atas menunjukkan proses pengurutan angka menggunakan algoritma Insertion Sort yang ditampilkan secara bertahap melalui GUI Java Swing. Pengguna memasukkan data angka 7, 1, 8, 24, 3, 25, 22, 9, 5, kemudian program menampilkan proses sorting pada bagian kanan aplikasi. Pada setiap langkah, program memperlihatkan angka yang sedang diproses dan hasil sementara setelah data ditempatkan pada posisi yang sesuai.

Misalnya pada Langkah 1, angka 1 dipindahkan ke depan karena lebih kecil dari 7, sehingga urutan berubah menjadi 1, 7, 8, 24, 3, 25, 22, 9, 5. Proses tersebut terus dilakukan pada angka berikutnya sampai seluruh data tersusun secara teratur. Dari visualisasi ini terlihat bahwa algoritma Insertion Sort bekerja dengan cara membandingkan data saat ini dengan data sebelumnya lalu menyisipkannya ke posisi yang tepat secara bertahap.

3.2 Saran Pengembangan

Program yang telah dibuat sebenarnya sudah berjalan dengan baik, tetapi masih bisa dikembangkan agar lebih menarik dan interaktif. Salah satu pengembangannya yaitu menambahkan pilihan metode sorting dalam satu aplikasi sehingga pengguna dapat membandingkan proses Insertion Sort, Selection Sort, dan Bubble Sort secara langsung. Selain itu, tampilan GUI juga bisa dibuat lebih menarik dengan penambahan warna berbeda saat proses pertukaran data berlangsung agar perpindahan elemen lebih mudah diamati.

Pengembangan lain yang dapat dilakukan yaitu menambahkan fitur pengurutan descending, penghitungan jumlah langkah atau waktu proses sorting, serta validasi input yang lebih lengkap. Dengan adanya fitur tersebut, program tidak hanya berfungsi sebagai latihan dasar sorting, tetapi juga dapat digunakan sebagai media pembelajaran untuk memahami efisiensi dan perbedaan performa setiap algoritma pengurutan.

DAFTAR PUSTAKA

- [1] A. H. Yunial, “Analisa Perbandingan Algoritma Bubble Sort, Insert Sort dan Selection Sort,” *Jurnal Informatika Universitas Pamulang*, vol. 9, no. 1, pp. 33–40, 2024. (openjournal.unpam.ac.id)
- [2] M. Dhamma, “Analisis Kompleksitas Diantara Algoritma Insertion Sort dan Selection Sort dan Diimplementasikan dengan Bahasa Pemrograman Java,” *InfoTekJar: Jurnal Nasional Informatika dan Teknologi Jaringan*, vol. 8, no. 1, pp. 6–9, 2024. (researchgate.net)
- [3] “Algoritma Pengurutan (Sorting): Bubble Sort, Selection Sort, Insertion Sort, Merge Sort,” YouTube, Mar. 20, 2024.

TUGAS PRAKTIKUM PEKAN 7

Pengurutan Nama Mahasiswa Secara Alfabetis Menggunakan GUI

1. Deskripsi Tugas

Program dikembangkan menggunakan Java GUI (Swing) untuk menampilkan proses pengurutan data mahasiswa secara interaktif.

Program menyediakan beberapa fitur utama, yaitu:

- Input data mahasiswa secara manual melalui form GUI.
- Menyimpan data mahasiswa ke dalam array atau ArrayList object.
- Pemilihan algoritma sorting menggunakan ComboBox.
- Menampilkan visualisasi proses sorting langkah demi langkah.
- Menampilkan hasil akhir pengurutan nama mahasiswa secara ascending (A-Z).

Perbandingan string dilakukan menggunakan method `compareToIgnoreCase()` agar proses sorting tidak membedakan huruf kapital dan huruf kecil.

2. Kode Program

2.1 Mahasiswa_2511531016

```
package pekan7_2511531016;

public class Mahasiswa_2511531016 {
    private String nama_1016;
    private String nim_1016;
    private String prodi_1016;

    public Mahasiswa_2511531016(
        String nama_1016,
        String nim_1016,
        String prodi_1016) {

        this.nama_1016 = nama_1016;
        this.nim_1016 = nim_1016;
        this.prodi_1016 = prodi_1016;
    }

    public String getNama_1016() {
        return nama_1016;
    }

    public void setNama_1016(String nama_1016) {
        this.nama_1016 = nama_1016;
    }
}
```

```

    public String getNim_1016() {
        return nim_1016;
    }

    public void setNim_1016(String nim_1016) {
        this.nim_1016 = nim_1016;
    }

    public String getProdi_1016() {
        return prodi_1016;
    }

    public void setProdi_1016(String prodi_1016) {
        this.prodi_1016 = prodi_1016;
    }

    @Override
    public String toString() {
        return nama_1016 + " - " + nim_1016 + " - " + prodi_1016;
    }
}

```

Penjelasan :

Program Mahasiswa_2511531016 merupakan class ADT (Abstract Data Type) yang digunakan untuk menyimpan data mahasiswa. Pada bagian awal program terdapat deklarasi atribut berupa nama_1016, nim_1016, dan prodi_1016 yang bertipe String. Ketiga atribut tersebut dibuat dengan keyword private agar data lebih aman dan tidak dapat diakses langsung dari luar class. Dengan konsep ini, data mahasiswa hanya dapat diakses melalui method tertentu seperti getter dan setter. Penggunaan ADT pada program ini bertujuan agar data mahasiswa tersusun lebih rapi dan mudah dikelola ketika digunakan pada proses sorting di GUI.

Selanjutnya, program memiliki constructor Mahasiswa_2511531016() yang berfungsi untuk menginisialisasi data mahasiswa saat object dibuat. Constructor menerima parameter nama, NIM, dan program studi, lalu menyimpannya ke dalam atribut class menggunakan keyword this. Selain constructor, terdapat juga method getter dan setter untuk masing-masing atribut. Method getter digunakan untuk mengambil nilai data mahasiswa, sedangkan setter digunakan untuk mengubah data mahasiswa jika diperlukan.

Dengan adanya getter dan setter, proses pengolahan data menjadi lebih fleksibel dan sesuai dengan konsep encapsulation dalam OOP.

Pada bagian akhir program terdapat method toString() yang berfungsi untuk mengubah object mahasiswa menjadi bentuk teks. Method ini akan menampilkan data mahasiswa dalam format nama, NIM, dan program studi dalam satu baris. Method toString() sangat membantu ketika object ingin ditampilkan ke tabel, console, atau JTextArea karena data dapat langsung terbaca dengan jelas. Secara keseluruhan, class Mahasiswa_2511531016 berfungsi sebagai tempat penyimpanan data utama yang nantinya digunakan oleh class GUI untuk proses input, tampilan data, dan pengurutan nama mahasiswa menggunakan algoritma sorting.

2.2 MahasiswaGUI_2511531016

```
package pekan7_2511531016;

import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.util.ArrayList;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingUtilities;
import javax.swing.table.DefaultTableModel;

public class MahasiswaGUI_2511531016 extends JFrame {
    private static final long serialVersionUID = 1L;
    private JTextField txtNama_1016;
    private JTextField txtNim_1016;
    private JTextField txtProdi_1016;
    private JButton btnTambah_1016;
    private JButton btnHapus_1016;
    private JButton btnSorting_1016;
    private JComboBox<String> comboSorting_1016;
    private JTable table_1016;
    private DefaultTableModel model_1016;
```



```

private JTextArea textArea_1016;
private ArrayList<Mahasiswa_2511531016> listMahasiswa_1016;

public MahasiswaGUI_2511531016() {
    setTitle("Sorting Nama Mahasiswa");
    setSize(1250, 700);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout(10, 10));

    listMahasiswa_1016 = new ArrayList<>();

    Font font_1016 = new Font("Arial", Font.PLAIN, 14);
    JPanel inputPanel_1016 = new JPanel();
    inputPanel_1016.setLayout(new FlowLayout(FlowLayout.LEFT, 10, 10));
    inputPanel_1016.setBorder(BorderFactory.createTitledBorder("Input Data Mahasiswa"));

    txtNama_1016 = new JTextField(10);
    txtNim_1016 = new JTextField(10);
    txtProdi_1016 = new JTextField(10);

    txtNama_1016.setFont(font_1016);
    txtNim_1016.setFont(font_1016);
    txtProdi_1016.setFont(font_1016);

    btnTambah_1016 = new JButton("Tambah Data");
    btnHapus_1016 = new JButton("Hapus Semua");
    btnSorting_1016 = new JButton("Mulai Sorting");
    btnTambah_1016.setPreferredSize(new Dimension(130, 30));
    btnHapus_1016.setPreferredSize(new Dimension(130, 30));
    btnSorting_1016.setPreferredSize(new Dimension(140, 30));

    comboSorting_1016 = new JComboBox<>();
    comboSorting_1016.addItem("Insertion Sort");

    comboSorting_1016.addItem("Selection Sort");
    comboSorting_1016.addItem("Bubble Sort");
    comboSorting_1016.setPreferredSize(new Dimension(160, 30));

    inputPanel_1016.add(new JLabel("Nama"));
    inputPanel_1016.add(txtNama_1016);
    inputPanel_1016.add(new JLabel("NIM"));
    inputPanel_1016.add(txtNim_1016);
    inputPanel_1016.add(new JLabel("Prodi"));
    inputPanel_1016.add(txtProdi_1016);
    inputPanel_1016.add(btnTambah_1016);
    inputPanel_1016.add(btnHapus_1016);
    inputPanel_1016.add(btnSorting_1016);
    inputPanel_1016.add(comboSorting_1016);
    add(inputPanel_1016, BorderLayout.NORTH);

    model_1016 = new DefaultTableModel();
    model_1016.addColumn("Nama");
    model_1016.addColumn("NIM");
    model_1016.addColumn("Program Studi");

    table_1016 = new JTable(model_1016);
    table_1016.setRowHeight(28);
    table_1016.setFont(font_1016);

    JScrollPane scrollTable_1016 = new JScrollPane(table_1016);
    scrollTable_1016.setBorder(BorderFactory.createTitledBorder("Data Mahasiswa"));
    add(scrollTable_1016, BorderLayout.CENTER);

    textArea_1016 = new JTextArea();
    textArea_1016.setEditable(false);
    textArea_1016.setFont(new Font("Monospaced", Font.PLAIN, 16));

```



```

JScrollPane scrollText_1016 = new JScrollPane(textArea_1016);
scrollText_1016.setBorder(BorderFactory.createTitledBorder("Visualisasi Sorting"));
scrollText_1016.setPreferredSize(new Dimension(1200, 200));
add(scrollText_1016, BorderLayout.SOUTH);

btnTambah_1016.addActionListener(e -> tambahData_1016());
btnHapus_1016.addActionListener(e -> hapusData_1016());
btnSorting_1016.addActionListener(e -> prosesSorting_1016());

setVisible(true);
}

private void tambahData_1016() {
    String nama_1016 = txtNama_1016.getText();
    String nim_1016 = txtNim_1016.getText();
    String prodi_1016 = txtProdi_1016.getText();
    if (nama_1016.isEmpty() || nim_1016.isEmpty() || prodi_1016.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Data tidak boleh kosong!");
        return;
    }
    Mahasiswa_2511531016 mhs_1016 = new Mahasiswa_2511531016(nama_1016, nim_1016, prodi_1016);
    listMahasiswa_1016.add(mhs_1016);
    model_1016.addRow(new Object[] {
        nama_1016, nim_1016, prodi_1016
    });

    txtNama_1016.setText("");
    txtNim_1016.setText("");
    txtProdi_1016.setText("");
}

private void hapusData_1016() {
    listMahasiswa_1016.clear();

    model_1016.setRowCount(0);
    textArea_1016.setText("");
}

private void prosesSorting_1016() {
    textArea_1016.setText("");
    String choice_1016 = comboSorting_1016.getSelectedItem().toString();
    if (choice_1016.equals("Insertion Sort")) {
        insertionSort_1016();
    }
    else if (choice_1016.equals("Selection Sort")) {
        selectionSort_1016();
    }
    else {
        bubbleSort_1016();
    }
    tampilData_1016();
}

private void insertionSort_1016() {
    textArea_1016.append("=== INSERTION SORT ===\n\n");
    for (int i_1016 = 1;
        i_1016 < listMahasiswa_1016.size();
        i_1016++) {
        Mahasiswa_2511531016 key_1016 = listMahasiswa_1016.get(i_1016);
        int j_1016 = i_1016 - 1;
        while (j_1016 >= 0 && listMahasiswa_1016.get(j_1016).getNama_1016().compareToIgnoreCase(key_1016.getNama_1016()) > 0) {
            listMahasiswa_1016.set(j_1016 + 1, listMahasiswa_1016.get(j_1016));
            j_1016--;
        }
        listMahasiswa_1016.set(j_1016 + 1, key_1016);
        textArea_1016.append("Langkah " + i_1016 + " : " + tampilNama_1016() + "\n\n");
    }
}

```

```

private void selectionSort_1016() {
    textArea_1016.append("=== SELECTION SORT ===\n\n");
    for (int i_1016 = 0; i_1016 < listMahasiswa_1016.size() - 1; i_1016++) {
        int minIndex_1016 = i_1016;
        for (int j_1016 = i_1016 + 1; j_1016 < listMahasiswa_1016.size(); j_1016++) {
            if (listMahasiswa_1016.get(j_1016).getNama_1016().compareToIgnoreCase(listMahasiswa_1016.get(minIndex_1016).getNama_1016()) < 0) {
                minIndex_1016 = j_1016;
            }
        }
        Mahasiswa_2511531016 temp_1016 = listMahasiswa_1016.get(i_1016);
        listMahasiswa_1016.set(i_1016, listMahasiswa_1016.get(minIndex_1016));
        listMahasiswa_1016.set(minIndex_1016, temp_1016);
        textArea_1016.append("Pass " + (i_1016 + 1) + " : " + tampilNama_1016() + "\n\n");
    }
}

private void bubbleSort_1016() {
    textArea_1016.append("=== BUBBLE SORT ===\n\n");
    for (int i_1016 = 0; i_1016 < listMahasiswa_1016.size() - 1; i_1016++) {
        for (int j_1016 = 0; j_1016 < listMahasiswa_1016.size() - i_1016 - 1; j_1016++) {
            if (listMahasiswa_1016.get(j_1016).getNama_1016().compareToIgnoreCase(listMahasiswa_1016.get(j_1016 + 1).getNama_1016()) > 0) {
                Mahasiswa_2511531016 temp_1016 = listMahasiswa_1016.get(j_1016);
                listMahasiswa_1016.set(j_1016, listMahasiswa_1016.get(j_1016 + 1));
                listMahasiswa_1016.set(j_1016 + 1, temp_1016);
            }
        }
        textArea_1016.append("Pass " + (i_1016 + 1) + " : " + tampilNama_1016() + "\n\n");
    }
}

private String tampilNama_1016() {
    String hasil_1016 = "[";
    for (int i_1016 = 0; i_1016 < listMahasiswa_1016.size(); i_1016++) {
        hasil_1016 += listMahasiswa_1016.get(i_1016).getNama_1016();

        if (i_1016 < listMahasiswa_1016.size() - 1) {
            hasil_1016 += ", ";
        }
    }
    hasil_1016 += "]";
    return hasil_1016;
}

private void tampilData_1016() {
    model_1016.setRowCount(0);
    for (Mahasiswa_2511531016 mhs_1016 : listMahasiswa_1016) {
        model_1016.addRow(new Object[] {
            mhs_1016.getNama_1016(),
            mhs_1016.getNim_1016(),
            mhs_1016.getProdi_1016()
        });
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new MahasiswaGUI_2511531016();
    });
}
}

```

Penjelasan :

Program MahasiswaGUI_2511531016 merupakan class utama berbasis Java Swing yang digunakan untuk membuat tampilan GUI pengurutan nama mahasiswa. Pada bagian awal program dilakukan import library Swing dan AWT seperti JFrame, JButton, JTable, JTextArea, JTextField, dan lainnya yang berfungsi untuk membangun tampilan antarmuka program. Class ini melakukan pewarisan terhadap JFrame sehingga program dapat ditampilkan dalam bentuk window desktop. Selain itu, terdapat deklarasi berbagai komponen GUI seperti text field untuk input data, tombol untuk menjalankan aksi, combo box untuk memilih algoritma

sorting, tabel untuk menampilkan data mahasiswa, dan text area untuk menampilkan visualisasi langkah sorting.

Pada constructor `MahasiswaGUI_2511531016()`, program mengatur ukuran frame, layout, serta membangun seluruh tampilan GUI. Panel input digunakan untuk memasukkan nama, NIM, dan program studi mahasiswa. Kemudian terdapat tombol Tambah Data, Hapus Semua, dan Mulai Sorting yang masing-masing memiliki fungsi berbeda. ComboBox digunakan untuk memilih algoritma sorting yang ingin digunakan, yaitu Insertion Sort, Selection Sort, atau Bubble Sort. Data mahasiswa yang dimasukkan akan disimpan ke dalam `ArrayList<Mahasiswa_2511531016>` sehingga data dapat dikelola secara dinamis. Selain itu, tabel (JTable) digunakan untuk menampilkan data mahasiswa, sedangkan JTextArea digunakan untuk memperlihatkan proses sorting langkah demi langkah agar pengguna dapat memahami alur pengurutan data.

Pada bagian method sorting, program menyediakan tiga algoritma pengurutan yaitu `insertionSort_1016()`, `selectionSort_1016()`, dan `bubbleSort_1016()`. Ketiga method tersebut melakukan pengurutan data berdasarkan nama mahasiswa menggunakan method `compareToIgnoreCase()` agar proses perbandingan huruf tidak membedakan huruf kapital dan kecil. Setiap kali terjadi proses sorting, hasil sementara akan ditampilkan pada JTextArea dalam bentuk langkah atau pass sehingga pengguna dapat melihat perubahan urutan data secara bertahap. Setelah proses sorting selesai, method `tampilData_1016()` akan memperbarui isi tabel sehingga data mahasiswa tampil dalam urutan alfabetis dari A-Z. Secara keseluruhan, program GUI ini tidak hanya menampilkan hasil akhir sorting, tetapi juga membantu memahami cara kerja algoritma sorting melalui visualisasi prosesnya secara langsung.

3. Output Kode Program

3.1 Output Insertion Sort

Nama	NIM	Program Studi
Kinaya	2511531016	Informatika
Dinda	2511531020	Informatika
Annisa	2511532024	Informatika
Halwa	2511531018	Informatika
Gina	2511533014	Informatika
Haliya	2511532002	Informatika

Visualisasi Sorting
=== INSERTION SORT ===

Langkah 1 : [Dinda, Kinaya, Annisa, Halwa, Gina, Haliya]

Langkah 2 : [Annisa, Dinda, Kinaya, Halwa, Gina, Haliya]

Langkah 3 : [Annisa, Dinda, Halwa, Kinaya, Gina, Haliya]

Langkah 4 : [Annisa, Dinda, Gina, Halwa, Kinaya, Haliya]

Langkah 5 : [Annisa, Dinda, Gina, Haliya, Halwa, Kinaya]

Penjelasan :

Output program di atas menunjukkan proses pengurutan nama mahasiswa menggunakan algoritma **Insertion Sort** melalui tampilan GUI Java Swing. Data mahasiswa yang dimasukkan ke dalam tabel terdiri dari Kinaya, Dinda, Annisa, Halwa, Gina, dan Haliya dengan urutan awal yang masih acak. Setelah tombol “Mulai Sorting” ditekan dan algoritma Insertion Sort dipilih, program mulai melakukan proses pengurutan secara bertahap berdasarkan alfabet. Pada bagian visualisasi sorting terlihat adanya Langkah 1 sampai Langkah 5 yang menunjukkan perubahan posisi nama mahasiswa selama proses sorting berlangsung. Setiap langkah memperlihatkan bagaimana satu data disisipkan ke posisi yang sesuai hingga urutannya menjadi lebih rapi.

Pada hasil akhirnya, data nama mahasiswa berhasil diurutkan menjadi Annisa, Dinda, Gina, Haliya, Halwa, dan Kinaya. Dari visualisasi tersebut terlihat bahwa Insertion Sort bekerja dengan cara membandingkan data saat

ini dengan data sebelumnya, lalu memindahkan posisi data jika urutannya belum sesuai. Proses perpindahan data terlihat bertahap dan cukup mudah dipahami karena perubahan urutannya tidak langsung sekaligus. Secara keseluruhan, program berhasil menampilkan cara kerja algoritma Insertion Sort dengan baik serta membantu memahami proses pengurutan data secara visual melalui GUI.

3.2 Output Selection Sort

Sorting Nama Mahasiswa

Input Data Mahasiswa

Nama NIM Prodi

Nama	NIM	Program Studi
Kinaya	2511531016	Informatika
Dinda	2511531020	Informatika
Annisa	2511532024	Informatika
Halwa	2511531018	Informatika
Gina	2511533014	Informatika
Haliya	2511532002	Informatika

Visualisasi Sorting

=== SELECTION SORT ===

Pass 1 : [Annisa, Dinda, Kinaya, Halwa, Gina, Haliya]

Pass 2 : [Annisa, Dinda, Kinaya, Halwa, Gina, Haliya]

Pass 3 : [Annisa, Dinda, Gina, Halwa, Kinaya, Haliya]

Pass 4 : [Annisa, Dinda, Gina, Haliya, Kinaya, Halwa]

Pass 5 : [Annisa, Dinda, Gina, Haliya, Halwa, Kinaya]

Penjelasan :

Output program di atas menunjukkan proses pengurutan nama mahasiswa menggunakan algoritma **Selection Sort**. Data awal mahasiswa yang dimasukkan ke dalam tabel masih dalam kondisi belum terurut, yaitu Kinaya, Dinda, Annisa, Halwa, Gina, dan Haliya. Setelah algoritma Selection Sort dipilih dan tombol “Mulai Sorting” ditekan, program mulai mencari nama dengan urutan alfabet paling kecil pada setiap proses pass. Hasil proses tersebut ditampilkan pada bagian visualisasi sorting dalam bentuk Pass 1,

Pass 2, hingga Pass 5 sehingga pengguna dapat melihat perubahan posisi data secara bertahap selama proses sorting berlangsung.

Dari visualisasi terlihat bahwa pada setiap pass, program memilih data terkecil lalu menukarnya dengan posisi awal yang sedang diperiksa. Proses ini terus dilakukan sampai seluruh nama mahasiswa tersusun menjadi urutan alfabet yang benar, yaitu Annisa, Dinda, Gina, Haliya, Halwa, dan Kinaya. Dibandingkan Bubble Sort, perpindahan data pada Selection Sort terlihat lebih sedikit karena pertukaran hanya dilakukan setelah data terkecil ditemukan. Secara keseluruhan, output program sudah berhasil menunjukkan cara kerja Selection Sort dengan jelas dan membantu pengguna memahami proses pengurutan data menggunakan metode pemilihan elemen terkecil.

3.3 Output Bubble Sort

Nama	NIM	Program Studi
Kinaya	2511531016	Informatika
Dinda	2511531020	Informatika
Annisa	2511532024	Informatika
Halwa	2511531018	Informatika
Gina	2511533014	Informatika
Haliya	2511532002	Informatika

Visualisasi Sorting

=== BUBBLE SORT ===

Pass 1 : [Dinda, Annisa, Halwa, Gina, Haliya, Kinaya]

Pass 2 : [Annisa, Dinda, Gina, Haliya, Halwa, Kinaya]

Pass 3 : [Annisa, Dinda, Gina, Haliya, Halwa, Kinaya]

Pass 4 : [Annisa, Dinda, Gina, Haliya, Halwa, Kinaya]

Pass 5 : [Annisa, Dinda, Gina, Haliya, Halwa, Kinaya]

Penjelasan :

Output program di atas menunjukkan proses pengurutan nama mahasiswa menggunakan algoritma **Bubble Sort**. Data mahasiswa yang

dimasukkan ke dalam tabel masih belum terurut, yaitu Kinaya, Dinda, Annisa, Halwa, Gina, dan Haliya. Setelah pengguna memilih Bubble Sort dan menekan tombol “Mulai Sorting”, program mulai membandingkan dua data yang berdekatan lalu menukarnya jika urutannya belum sesuai alfabet. Pada bagian visualisasi sorting terlihat adanya Pass 1 sampai Pass 5 yang menunjukkan perubahan posisi nama mahasiswa secara bertahap selama proses pengurutan berlangsung.

Pada proses Bubble Sort terlihat bahwa nama dengan urutan alfabet lebih besar akan terus bergerak ke belakang pada setiap pass sampai seluruh data tersusun dengan benar. Dari hasil akhirnya, data berhasil diurutkan menjadi Annisa, Dinda, Gina, Haliya, Halwa, dan Kinaya. Dibandingkan algoritma lainnya, Bubble Sort melakukan lebih banyak proses perbandingan sehingga perubahan posisi data terlihat lebih sering pada visualisasi. Walaupun prosesnya cukup panjang, algoritma ini mudah dipahami karena hanya membandingkan data yang saling berdekatan. Secara keseluruhan, program berhasil menjalankan Bubble Sort dengan baik dan mampu menampilkan proses pengurutan data secara jelas melalui GUI.